

Architecting efficient context-aware multi-agent framework for production

Hangfei Lin

The landscape of AI agent development is shifting fast. We've moved beyond prototyping single-turn chatbots. Today, organizations are deploying sophisticated, autonomous agents to handle **long-horizon tasks**: automating workflows, conducting deep research, and maintaining complex codebases.

That ambition immediately runs into a bottleneck: **context**.

As agents run longer, the amount of information they need to track—chat history, tool outputs, external documents, intermediate reasoning—**explodes**. The prevailing “solution” has been to lean on ever-larger context windows in foundation models. But simply giving agents more space to paste text can not be the single scaling strategy.

To build production-grade agents that are **reliable, efficient, and debuggable**, the industry is exploring a new discipline:

Context engineering — treating context as a first-class system with its own architecture, lifecycle, and constraints.

Based on our experience scaling complex single- or multi-agentic systems, we designed and evolved the context stack in [Google Agent Development Kit \(ADK\)](#) to support that discipline. ADK is an open-source, multi-agent-native framework built to make **active context engineering** achievable in real systems.

The scaling bottleneck

A large context window will help context-related problems but won't address all context-related problems. In practice, the naive pattern—**append everything into one giant prompt**—collapses under a three-way pressure:

1. **Cost and latency spirals**: Model cost and time-to-first-token grow quickly with context size. "Shoveling" raw history and verbose tool payloads into the window makes agents prohibitively slow and expensive.
2. **Signal degradation (“lost in the middle”)**: A context window flooded with irrelevant logs, stale tool outputs, or deprecated state can distract the model, causing it to fixate on past patterns rather than the immediate instruction. To ensure robust decision-making, we must maximize the density of relevant information.
3. **Physical limits**: Real-world workloads—involving full RAG results, intermediate artifacts, and long conversation traces—eventually overflow even the largest fixed windows.

Throwing more tokens at the problem buys time, but it doesn't change the shape of the curve. To scale, we need to change **how context is represented and managed**, not just how much of it we can cram into a single call.

The design thesis: context as a compiled view

In the previous generation of agent frameworks, context was treated like a mutable string buffer. ADK is built around a different thesis: **Context is a compiled view over a richer stateful system.**

In that view:

- **Sessions, memory, and artifacts (files)** are the *sources*— the full, structured state of the interaction and its data.
- **Flows and processors** are the *compiler pipeline* — a sequence of passes that transform that state.
- The **working context** is the *compiled view* you ship to the LLM for this one invocation.

Once you adopt this mental model, context engineering stops being prompt gymnastics and starts looking like systems engineering. You are forced to ask standard systems questions: *What is the intermediate representation? Where do we apply compaction? How do we make transformations observable?*

ADK's architecture answers these questions via three design principles:

- **Separate storage from presentation:** We distinguish between durable state (Sessions) and per-call views (working context). This allows you to evolve storage schemas and prompt formats independently.
- **Explicit transformations:** Context is built through named, ordered processors, not ad-hoc string concatenation. This makes the "compilation" step observable and testable.
- **Scope by default:** Every model call and sub-agent sees the minimum context required. Agents must reach for more information explicitly via tools, rather than being flooded by default.

ADK's tiered structure, its relevance mechanisms, and its multi-agent handoff semantics—is essentially an application of this "compiler" thesis and these three principles:

- **Structure** — a tiered model that separates how information is stored from what the model sees.
- **Relevance** — agentic and human controls that decide what matters *now*.
- **Multi-agent context** — explicit semantics for handing off the right slice of context between agents.

The next sections walk through each of these pillars in turn.

1. Structure: The tiered model

Most early agent systems implicitly assume a **single window of context**. ADK goes the other way. It separates storage from presentation and organizes context into distinct layers, each with a specific job:

- **Working context** — the immediate prompt for *this* model call: system instructions, agent identity, selected history, tool outputs, optional memory results, and references to artifacts.
- **Session** — the **durable log** of the interaction: every user message, agent reply, tool call, tool result, control signal, and error, captured as structured Event objects.
- **Memory** — long-lived, **searchable knowledge** that outlives a single session: user preferences, and past conversations.

- **Artifacts** – large binary or textual data associated with the session or user (files, logs, images), addressed by name and version rather than pasted into the prompt.

1.1 Working context as a recomputed view

For each invocation, ADK rebuilds the Working Context from the underlying state. It starts with instructions and identity, pulls in selected Session events, and optionally attaches memory results. This view is **ephemeral** (thrown away after the call), **configurable** (you can change formatting without migrating storage), and **model-agnostic**.

This flexibility is the first win of the compiler view: you stop hard-coding "the prompt" and start treating it as a derived representation you can iterate on.

1.2 Flows and processors: context processing as a pipeline

Once you separate storage from presentation, you need machinery to "compile" one into the other. In ADK, every LLM-based agent is backed by an **LLM Flow**, which maintains ordered lists of processors.

A (simplified) SingleFlow might look like:

These flows are ADK's machinery to compile context. The order matters: each processor builds on the outputs of the previous steps. This gives you natural insertion points for custom filtering, compaction strategies, caching, and multi-agent routing. You are no longer rewriting giant "prompt templates"; you're just adding or reordering processors.

1.3 Session and events: structured, language-agnostic history

An ADK Session represents the definitive state of a conversation or workflow instance. Concretely, it acts as a container for session metadata (IDs, app names), a state scratchpad for structured variables, and—most importantly—a chronological list of **Events**.

Instead of storing raw prompt strings, ADK captures every interaction—user messages, agent replies, tool calls, results, control signals, and errors—as strongly-typed Event records. This structural choice pays three distinct advantages:

- **Model agnosticism:** You can swap underlying models without rewriting the history, as the storage format is decoupled from the prompt format.
- **Rich operations:** Downstream components like compaction, time-travel debugging, and memory ingestion can operate over a rich event stream rather than parsing opaque text.
- **Observability:** It provides a natural surface for analytics, allowing you to inspect precise state transitions and actions.

The bridge between this session and the working context is the `contents` processor. It performs the heavy lifting of transforming the Session into the history portion of the working context by executing three critical steps:

- **Selection:** It filters the event stream to drop irrelevant events, partial events, and framework noise that shouldn't reach the model.
- **Transformation:** It flattens the remaining events into `Content` objects with the correct roles (user/assistant/tool) and annotations for the specific model API being used.
- **Injection:** It writes the formatted history into `llm_request.contents`, ensuring downstream

processors—and the model itself—receive a clean, coherent conversational trace.

In this architecture, the Session is your **ground truth**; the working context is merely a **computed projection** that you can refine and optimize over time.

1.4 Context compaction and filtering at the session layer

If you keep appending raw events indefinitely, latency and token usage will inevitably spiral out of control. ADK's Context Compaction feature attacks this problem at the Session layer.

When a configurable threshold (such as the number of invocations) is reached, ADK triggers an asynchronous process. It uses an LLM to summarize older events over a sliding window—defined by compaction intervals and overlapping size—and writes the resulting summary back into the Session as a new event with a "compaction" action. Crucially, this allows the system to prune or deprioritize the raw events that were summarized.

Because compaction operates on the Event stream itself, the benefits cascade downstream:

- **Scalability:** Sessions remain physically manageable even for extremely long-running conversations.
- **Clean views:** The contents processor automatically works over a history that is already compacted, requiring no complex logic at query time.
- **Decoupling:** You can tune compaction prompts and strategies without touching a single line of agent code or template logic.

This creates a scalable lifecycle for long contexts. For strictly rule-based reduction, ADK offers a sibling operation—**Filtering**—where prebuilt plugins can globally drop or trim context based on deterministic rules before it ever reaches the model.

1.5 Context caching

Modern models support **context caching** (prefix caching), which allows the inference engine to reuse attention computation across calls. ADK's separation of "Session" (storage) and "Working Context" (view) provides a natural substrate for this optimization.

The architecture effectively divides the context window into two zones:

- **Stable prefixes:** System instructions, agent identity, and long-lived summaries.
- **Variable suffixes:** The latest user turn, new tool outputs, and small incremental updates.

Because ADK flows and processors are explicit, you can treat cache-friendliness as a hard design constraint. You can order your pipeline to keep frequently reused segments stable at the front of the context window, while pushing highly dynamic content toward the end. To enforce this rigor, we introduced `static instruction`, a primitive that guarantees immutability for system prompts, ensuring that the cache prefix remains valid across invocations.

This is a prime example of context engineering acting as systems work across the full stack: you are not only deciding what the model sees, but optimizing how often the hardware has to re-compute the underlying tensor operations.

2. Relevance: Agentic management of what matters now

Once the structure is established, the core challenge shifts to relevance: *Given a tiered context*

architecture, what specific information belongs in the model's active window right now?

ADK answers this through a collaboration between human domain knowledge and agentic decision-making. Relying solely on hard-coded rules is cost-effective but rigid; relying solely on the agent to browse everything is flexible but prohibitively expensive and unstable.

An optimal Working Context is a negotiation between the two. Human engineers define the architecture—where data lives, how it is summarized, and what filters apply. The Agent then provides the intelligence, deciding dynamically when to "reach" for specific memory blocks or artifacts to satisfy the immediate user request.

2.1 Artifacts: externalizing large state

Early agent implementations often fall into the "context dumping" trap: placing large payloads—a 5MB CSV, a massive JSON API response, or a full PDF transcript—directly into the chat history. This creates a permanent tax on the session; every subsequent turn drags that payload along, burying critical instructions and inflating costs.

ADK solves this by treating large data as **Artifacts**: named, versioned binary or text objects managed by an `ArtifactService`.

Conceptually, ADK applies a **handle pattern** to large data. Large data lives in the artifact store, not the prompt. By default, agents see only a lightweight reference (a name and summary) via the request processor. When—and only when—an agent requires the raw data to answer a question, it uses the `LoadArtifactsTool`. This action temporarily loads the content into the Working Context.

Crucially, ADK supports **ephemeral expansion**. Once the model call or task is complete, the artifact is offloaded from the working context by default. This turns "5MB of noise in every prompt" into a precise, on-demand resource. The data can be huge, but the context window remains lean.

2.2 Memory: long-term knowledge, retrieved on demand

Where Artifacts handle discrete, large objects, ADK's **Memory** layer manages long-lived, semantic knowledge that extends beyond a single session—user preferences, past decisions, and domain facts.

We designed the `MemoryService` around two principles: memory must be **searchable** (not permanently pinned), and retrieval should be **agent-directed**.

The `MemoryService` ingests data—often from finished Sessions—into a vector or keyword corpus. Agents then access this knowledge via two distinct patterns:

- **Reactive recall:** The agent recognizes a knowledge gap ("What is the user's dietary restriction?") and explicitly calls the `load_memory_tool` to search the corpus.
- **Proactive recall:** The system uses a pre-processor to run a similarity search based on the latest user input, injecting likely relevant snippets via the `preload_memory_tool` before the model is even invoked.

This approach replaces the "context stuffing" anti-pattern with a "memory-based" workflow. Agents recall exactly the snippets they need for the current step, rather than carrying the weight of every conversation they have ever had.

3. Multi-agent context: who sees what, when

Single-agent systems struggle with context bloat; multi-agent systems amplify it. If a root agent passes its full history to a sub-agent, and that sub-agent does the same, you trigger a **context explosion**. The token count skyrockets, and sub-agents get confused by irrelevant conversational history.

Whenever an agent invokes another agent, ADK lets you **explicitly scope** what the callee sees—maybe just the latest user query and one artifact—while suppressing most of the ancestral history.

3.1 Two multi-agent interaction patterns

At a high level, ADK maps multi-agent interactions into two distinct architectural patterns.

The first is Agents as Tools. Here, the root agent treats a specialized agent strictly as a function: call it with a focused prompt, get a result, and move on. The callee sees only the specific instructions and necessary artifacts—no history.

The second is Agent Transfer (Hierarchy). Here, control is fully handed off to a sub-agent to continue the conversation. The sub-agent inherits a view over the Session and can drive the workflow, calling its own tools or transferring control further down the chain.

3.2 Scoped handoffs for agent transfer

Handoff behavior is controlled by knobs like `include_contents` on the callee, which determine how much context flows from the root agent to a sub-agent. In the default mode, ADK passes the full contents of the caller's working context—useful when the sub-agent genuinely benefits from the entire history. In none mode, the sub-agent sees no prior history; it only receives the new prompt you construct for it (for example, the latest user turn plus a couple of tool calls and responses). Specialized agents get the minimal context they need, rather than inheriting a giant transcript by default.

Because a sub-agent's context is also built via processors, these handoff rules plug into the same flow pipeline as single-agent calls. You don't need a separate multi-agent machinery layer; you're just changing how much upstream state the existing context compiler is allowed to see.

3.3 Translating conversations for agent transfer

Foundation models operate on a fixed role schema: `system`, `user`, and `assistant`. They do not natively understand "Assistant A" vs. "Assistant B."

When ADK transfers control, it must often **reframe** the existing conversation so the new agent sees a coherent working context. If the new agent simply sees a stream of "Assistant" messages from the previous agent, it will hallucinate that *it* performed those actions.

To prevent this, ADK performs an active translation during handoff:

- **Narrative casting:** Prior "Assistant" messages may be re-cast as narrative context (e.g., modifying the role or injecting a tag like `[For context]: Agent B said...`) rather than appearing as the new agent's own outputs.
- **Action attribution:** Tool calls from other agents are marked or summarized so the new agent acts on the *results* without confusing the *execution* with its own capabilities.

Effectively, ADK builds a fresh Working Context from the sub-agent's point of view, while preserving the factual history in the Session. This ensures correctness, allowing each agent to

assume the "Assistant" role without misattributing the broader system's history to itself.

Conclusion

As we push agents to tackle longer horizons, "context management" can no longer mean "string manipulation." It must be treated as an architectural concern alongside storage and compute.

ADK's context architecture—tiered storage, compiled views, pipeline processing, and strict scoping—is our answer to this challenge. It encapsulates the rigorous systems engineering required to move agents from interesting prototypes to scalable, reliable production systems.